Introdução a Linguagens e Paradigmas de Programação

Prof. Jônata Tyska Carvalho



Por que estudar estes conceitos?

- Melhor capacidade para:
  - Expressar ideias (soluções)
  - Escolher linguagens apropriadas
  - Aprender novas linguagens
  - Entender especificidades de implementação
  - Explorar linguagens já conhecidas
  - Avanço geral da computação



Domínios

- Computação Científica
  - Estruturas de dados simples - vetores e matrizes
  - Muitas operações de ponto de flutuante
  - Fortran e ALGOL 60

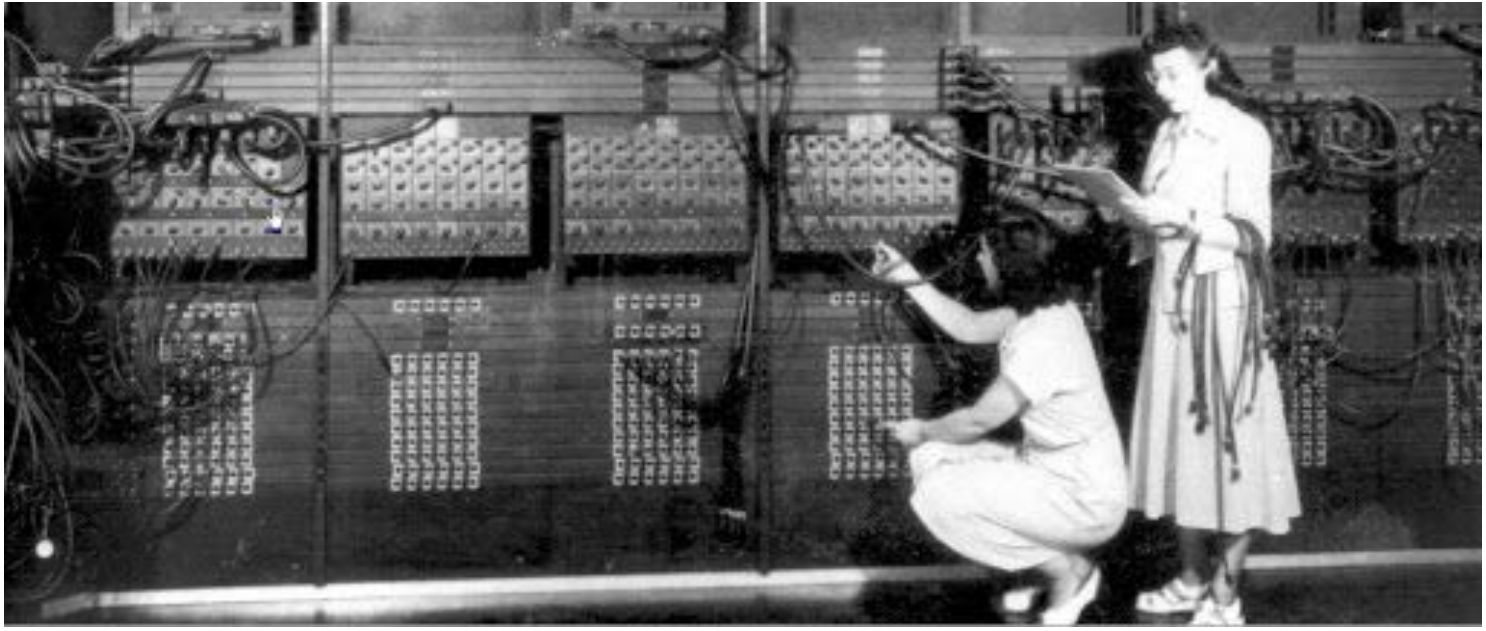
- Aplicações Empresariais
  - Relatórios, números decimais e caracteres
  - COBOL

Domínios

- Inteligência Artificial
  - Lida com símbolos no lugar de números
  - Usa listas ligadas ao invés de vetores
  - LISP: funcional, PROLOG: lógica
- Sistemas para a WEB
  - Coleção de linguagens utilizadas
  - Fornecer conteúdo dinâmico
  - JavaScript, PHP, Java, Python (HTML, CSS)

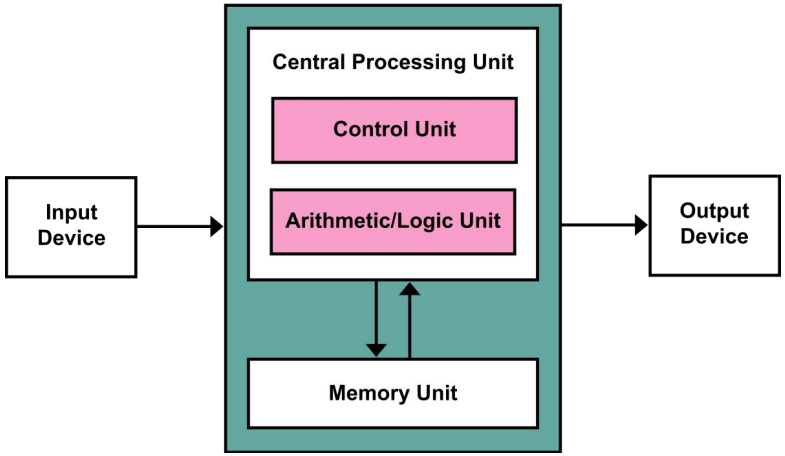
Histórico

Programar = tarefa lenta, tediosa e inflexível



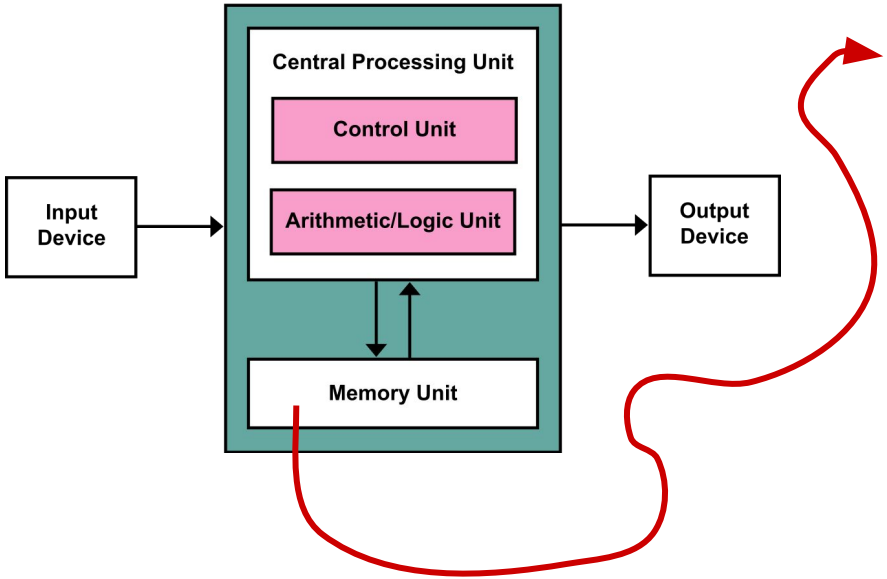
Histórico

Programa armazenado - arquitetura de Von Neumann



Histórico

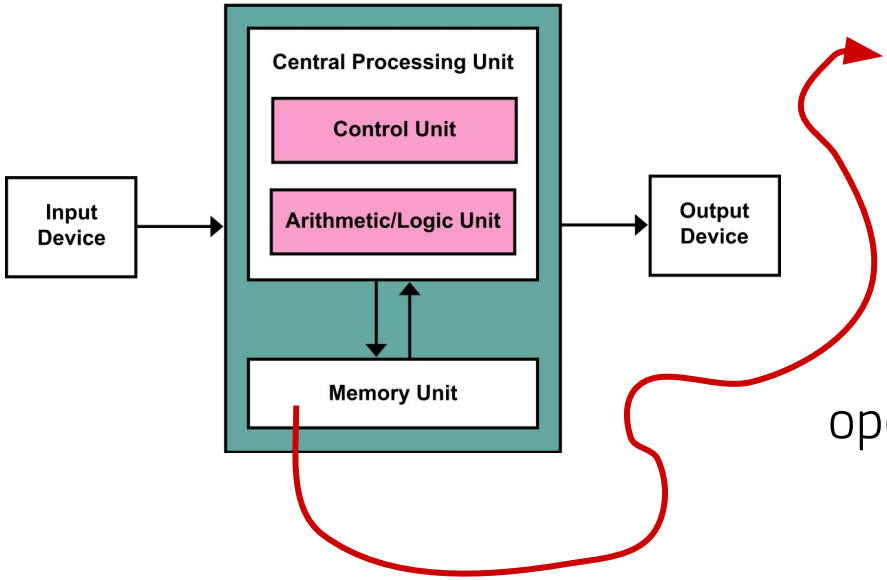
Programa armazenado - arquitetura de Von Neumann



Armazenado em que formato?

Histórico

Programa armazenado - arquitetura de Von Neumann



Armazenado em que formato?

Binário

0110 1100 0101

opcode end1 end2

Histórico Primeira Geração

Código de máquina (*Machine code*)

```
0101 0101 0110 1100
1010 0110 0110 1101
1100 0000 0101 0110
0011 0000 0110 1110
1111 0000 0000 0000
```

Código para carregar dois números da memória, somá-los, e armazenar o resultado em um terceiro endereço de memória

Dependente da máquina (conjunto de instruções)

Histórico Segunda Geração

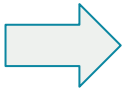
Linguagem de Montagem (*Assembly*)

Usa mnemônicos

```
MOV R5, 6C
LD R6, 6D
ADDI R0, R5 R6
ST R0, 6E
HLT
```



ASSEMBLER



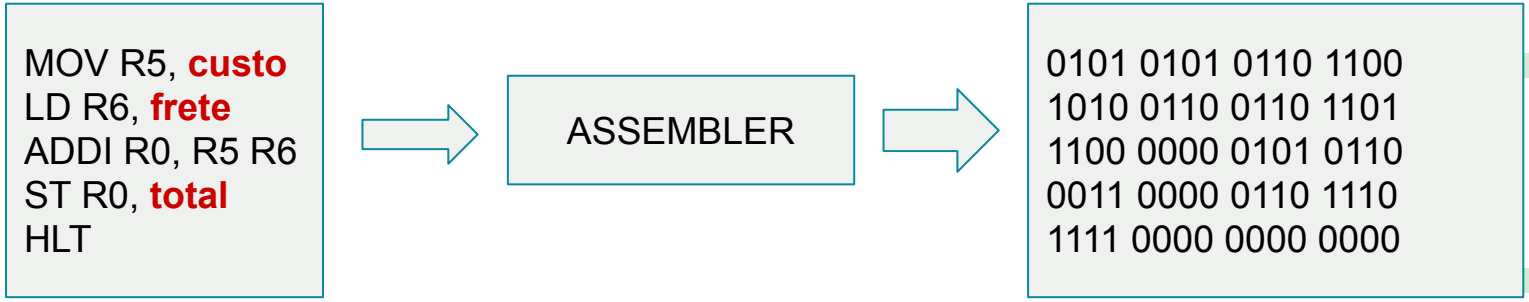
```
0101 0101 0110 1100
1010 0110 0110 1101
1100 0000 0101 0110
0011 0000 0110 1110
1111 0000 0000 0000
```

Código de máquina

Histórico Segunda Geração

Linguagem de Montagem (*Assembly*)

Usa mnemônicos e **identificadores**



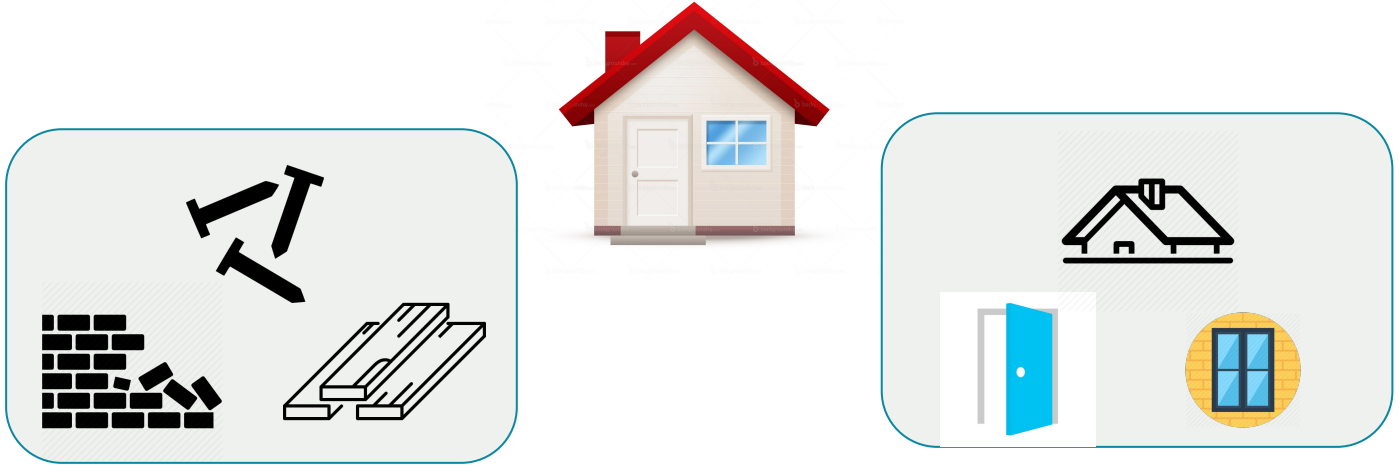
Código de máquina

Assembly ainda **dependente da máquina** (ou família)

Histórico Segunda Geração

Linguagem de Montagem (*Assembly*)

- Programador pode escrever com mnemônicos
- Porém ainda em passos pequenos e incrementais
- Analogia: projetar casa em termos de pregos, tábuas e tijolos



Histórico Terceira Geração

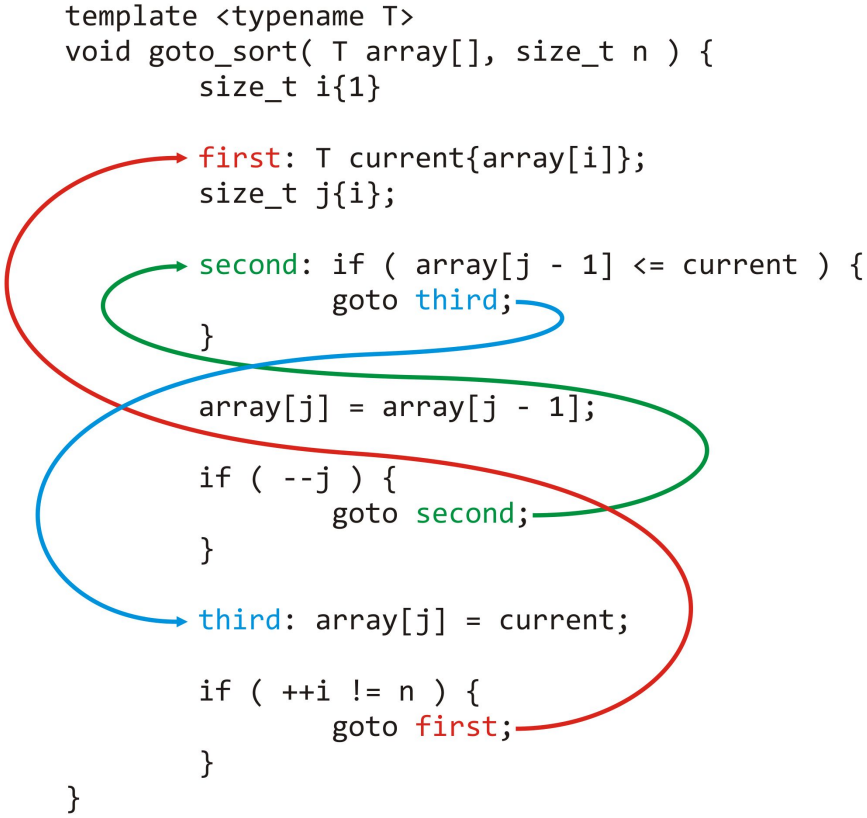
Linguagens de mais alto nível

- Independentes de máquina
- Primitivas de alto nível **traduzidas** para *machine code*
- **Compilar** diversas instruções de máquina para representar instruções de alto nível
- Outra forma: **interpretação**
- FORTRAN (Fórmula Translator) - científica
- COBOL (U.S. Navy) - orientada a negócio

Histórico Terceira Geração

- Dijkstra = programação estruturada
  - **GOTO is evil!**
  - Sequência, seleção e repetição

    Böhn e Jacopini
- Dividir e conquistar



Histórico Outras Gerações

- 1GL: Linguagem de máquina: binária
- 2GL: Montagem: *assembly*
- 3GL: Estruturadas: Fortran, ALGOL, COBOL, C...
- 4GL: Orientadas à aplicação - alto-nível com objetivos específicos: SQL, Lotus,...
- 5GL: Linguagens lógicas e funcionais: prolog, lisp,...

P.S.: A classificação das linguagens de programação em gerações é uma questão que apresenta divergências de autor para autor. Apresentada aqui Bal, Henri E.; Grune, Dick. Programming Language Essentials. Wokingham: Addison-Wesley, 1994.

Paradigmas

- O que é um paradigma?

  "(...) padrão de resolução de problemas que se relaciona a um determinado gênero de programas e linguagens."

  [Tucker, Noonan, 2009]

  Modelo, padrão ou estilo de programação suportado por linguagens que agrupam certas características comuns.

Paradigmas

- **Imperativo:** abordagem tradicional, sequência de comandos que **manipula dados** para produzir resultado
  - Define **como** fazer

- **Declarativo:** abordagem em que o programador define **o que** precisa ser feito
  - Construção de programas através de relações ou funções

Paradigmas imperativos

- Podem ser divididos em:
  - **Programação Procedural**
    - Baseado no conceito de chamadas a procedimento
    - Procedimento = rotinas, sub-rotinas, funções contendo conjuntos de instruções
  - **Programação Orientada a Objetos**
    - Conjunto de objetos interagindo
    - Dados e operações são partes dos objetos

Programação Procedural

Procedimentos e funções

```
PROGRAM MaiorN1;
VAR
  Num1,Num2,Num3,Maior : Integer;
  PROCEDURE LeNum(VAR N1: Integer; VAR N2: Integer; VAR N3: Integer);
  BEGIN
      write('Entre com o primeiro numero -> '); readln(N1);
      write('Entre com o segundo numero -> '); readln(N2);
      write('Entre com o terceiro numero -> '); readln(N3);
  END; (* LeNum *)
  FUNCTION DetMaior(N1:Integer;N2:Integer;N3:Integer): Integer;
      var M: Integer;
  BEGIN
      M := Num1;
      IF M < Num2 THEN M := Num2;
     IF M < Num3 THEN M := Num3;
      DetMaior:=M;
  END; (* DetMaior *)
```

Chamadas

```
BEGIN (* Programa Principal *)
  LeNum(Num1,Num2,Num3);
  Maior:=DetMaior(Num1,Num2,Num3);
  ImpMaior(Maior);
END.
```

Linguagens procedurais



Programação orientada a objetos

- Sistema de software é uma coleção de objetos
- **Abstrações** do mundo real (ou do problema)
- Objetos são entidades independentes que possuem uma identidade, estado e comportamento próprios
- Dados estão **encapsulados** nos objetos

Programação orientada a objetos

- Objetos realizam **ações** sob si próprios e solicitam execução de ações de outros objetos

- As interações entre objetos garantem a solução de um problema

- As características (dados e métodos) dos objetos são definidas através de **classes**



Programação orientada a objetos

```java
public class Termometro {

    private float temperaturaCelsius;

    public Termometro (float temperatura){

        this.temperaturaCelsius = temperatura;
    }

    public void setTemperatura (float temperatura){
        this.temperaturaCelsius = temperatura;
    }

    public float getTemperatura (){
        return this.temperaturaCelsius;
    }

    public void resfriarTermometro(){
        this.temperaturaCelsius = temperaturaCelsius - 5;
    }

    public void aquecerTermometro(){
        this.temperaturaCelsius = temperaturaCelsius + 5;
    }

    public float converterTemperaturaFahrenheit (){

       return (float) ((this.temperaturaCelsius * 1.8) + 32);

    }

}
```

Classe

Objeto

```java
public static void main(String[] args) {

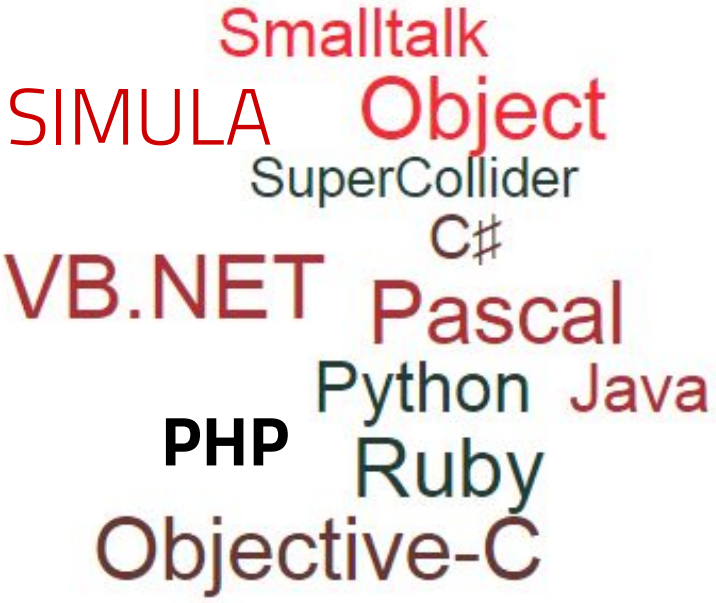
    Termometro meuTermometro;
    meuTermometro = new Termometro(15);

    meuTermometro.aquecerTermometro();
    meuTermometro.aquecerTermometro();

    System.out.println("Celsius: " + meuTermometro.getTemperatura());
    System.out.println("Fahrenheit: " + meuTermometro.converterTemperaturaFahrenheit());

    meuTermometro.resfriarTermometro();
    System.out.println("Celsius: " + meuTermometro.getTemperatura());
    System.out.println("Fahrenheit: " + meuTermometro.converterTemperaturaFahrenheit());
}
```

Linguagens orientadas a objetos



Paradigmas declarativos

- Podem ser divididos em:
  - **Programação funcional**
    - Trata computação como avaliação de funções
    - Evita estados ou dados mutáveis
  - **Programação lógica**
    - Programas definidos através de relações
    - Faz uso da **lógica matemática**

Programação funcional

Em linguagem C:

```
total = 0;
for (i=1; i<=10; ++i)
   total = total + i;
```

Em Haskell:

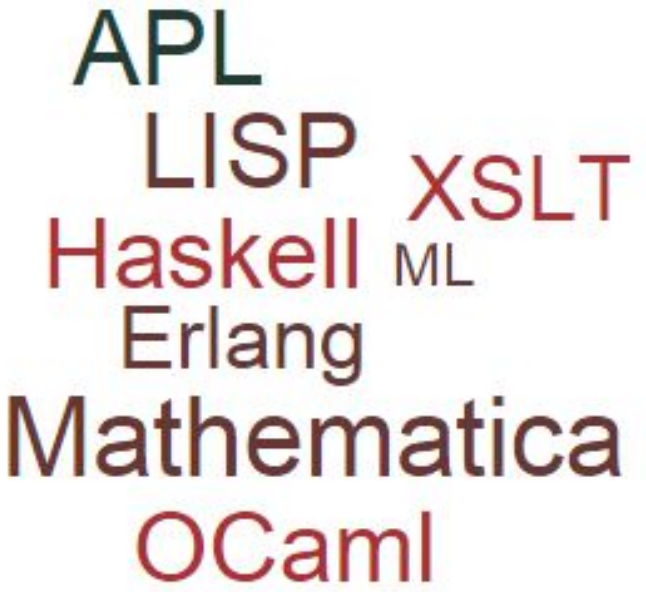
```
sum [1..10]
```

Consiste em aplicar a função *sum* à lista dos inteiros entre 1 e 10

Programação funcional

- Sem efeitos colaterais
- Ideal para programação concorrente
- Programas construídos através de combinação de funções
- Estados (re)construídos a partir da execução de múltiplas operações - exemplo: saldo da conta

Linguagens funcionais



Programação lógica

Exemplo em Prolog

Fatos / Base de conhecimento

```
come(urso, peixe).
come(peixe, peixinho).
come(peixinho, alga).
come(quati, peixe).
come(urso, quati).
come(urso, raposa).
come(raposa, coelho).
come(coelho, mato).
come(urso, cavalo).
come(cavalo, mato).
come(gato-selvagem, cavalo).

animal(urso).
animal(peixe).
animal(peixinho).
animal(quati).
animal(raposa).
animal(coelho).
animal(cavalo).
animal(gato-selvagem).

planta(mato).
planta(alga).
```

Programação lógica

Exemplo em Prolog

Fatos / Base de conhecimento

```
come(urso, peixe).
come(peixe, peixinho).
come(peixinho, alga).
come(quati, peixe).
come(urso, quati).
come(urso, raposa).
come(raposa, coelho).
come(coelho, mato).
come(urso, cavalo).
come(cavalo, mato).
come(gato-selvagem, cavalo).

animal(urso).
animal(peixe).
animal(peixinho).
animal(quati).
animal(raposa).
animal(coelho).
animal(cavalo).
animal(gato-selvagem).

planta(mato).
planta(alga).
```

Regras

```
presa(X):-come(Y, X), animal(X).
caçado(X):- presa(X).
```

Programação lógica

Exemplo em Prolog

Fatos / Base de conhecimento

```
come(urso, peixe).
come(peixe, peixinho).
come(peixinho, alga).
come(quati, peixe).
come(urso, quati).
come(urso, raposa).
come(raposa, coelho).
come(coelho, mato).
come(urso, cavalo).
come(cavalo, mato).
come(gato-selvagem, cavalo).

animal(urso).
animal(peixe).
animal(peixinho).
animal(quati).
animal(raposa).
animal(coelho).
animal(cavalo).
animal(gato-selvagem).

planta(mato).
planta(alga).
```

Regras

```
presa(X):-come(Y, X), animal(X).
caçado(X):- presa(X).
```

Consultas e respostas

```
?- animal(coelho).
yes

?- come(gato-selvagem, mato).
no

?- come(X, peixe).
Urso;
Quati.

?- come(X,Y), planta(Y).
peixinho alga;
coelho mato;
cavalo mato.
```

Linguagens lógicas



Lado a Lado: Calculando fatorial

Paradigma Imperativo

Exemplo: Fatorial em C

```
int fatorial (int n)
{
    int i;
    for (i = n - 1; i >= 1; i-)
       n = n * i;
    return n;
}
```

Paradigma Funcional

Exemplo: Fatorial em LISP

```
(defun fatorial (n)
    (if (<= n 1)
     1
     (* n (fatorial (- n 1))))
    )
)
```

Paradigma Lógico

Exemplo: Fatorial em Prolog

```
fatorial(0,1).
fatorial(N,R) :- N1 is N - 1, fatorial(N1, R1), R
is N * R1.
```

Critérios de avaliação de linguagens

Tabela 1.1 Critérios de avaliação de linguagens e as características que os afetam

	CRITÉRIOS		
Característica	**LEGIBILIDADE**	**FACILIDADE DE ESCRITA**	**CONFIABILIDADE**
Simplicidade	•	•	•
Ortogonalidade	•	•	•
Tipos de dados	•	•	•
Projeto de sintaxe	•	•	•
Suporte para abstração		•	•
Expressividade		•	•
Verificação de tipos			•
Tratamento de exceções			•
Apelidos restritos			•

(SEBESTA, 2010)

Critérios de avaliação de linguagens

- **Legibilidade:** facilidade com a qual os programas podem ser lidos e entendidos
- **Facilidade de escrita:** facilidade com a qual uma linguagem pode ser usada para criar programas para um dado domínio
- **Confiabilidade:** conformidade com as especificações

Critérios de avaliação de linguagens

- Simplicidade e Ortogonalidade
  - Um conjunto controlável de recursos e construções
  - Poucas construções, número pequeno de primitivas e um pequeno conjunto de regras para combiná-las
- Tipos de dados
  - Mecanismos adequados para definir tipos de dados
- Projeto da sintaxe
  - Formato dos identificadores
  - Palavras especiais e métodos de formar sentenças compostas
  - Forma e significado: construções autodescritivas, palavras-chave significativas

Critérios de avaliação de linguagens

- Suporte à abstração
  - A habilidade de definir e usar estruturas ou operações complicadas de forma a permitir que muitos dos detalhes sejam ignorados
- Expressividade
  - Um conjunto de formas relativamente convenientes de especificar as operações
- Verificação de tipos
  - Testes para detectar erros de tipos
- Tratamento de exceções
  - Interceptar erros em tempo de execução e tomar medidas corretivas
- Apelidos restritos
  - Poucas maneiras de expressar a mesma idéia

Outros critérios de avaliação

- **Custo**
  - Treinar programadores para usar a linguagem
  - Compilar e executar programas
  - Sistema de implementação da linguagem: disponibilidade de compiladores gratuitos
- **Portabilidade**
  - A facilidade com a qual os programas podem ser movidos de uma plataforma para outra
- **Generalidade**
  - A aplicabilidade a uma ampla faixa de aplicações
- **Bem definida**
  - Em relação à completude e à precisão do documento oficial que define a linguagem

Classificação quanto ao **grau de abstração**

- **Baixo nível**
  - Símbolos são uma representação direta do código de máquina que será gerado.
    - Exemplo: Assembly.
- **Médio nível**
  - símbolos podem ser convertidos diretamente para código de máquina.
    - Exemplo: C, C++.
- **Alto nível**
  - símbolos mais complexos, inteligível pelo ser humano e não-executável diretamente pela máquina
    - Exemplo: SQL.

Classificação quanto à **estrutura de tipos**

- Fracamente tipada
  - o tipo da variável muda dinamicamente conforme a situação.
  - Exemplo: PHP e Smalltalk

- Fortemente tipada
  - o tipo da variável, uma vez atribuído, se mantém o mesmo até ser descartada da memória.
  - Exemplo: Java, Python e Ruby

Linguagens de Programação

Classificação quanto à **execução**

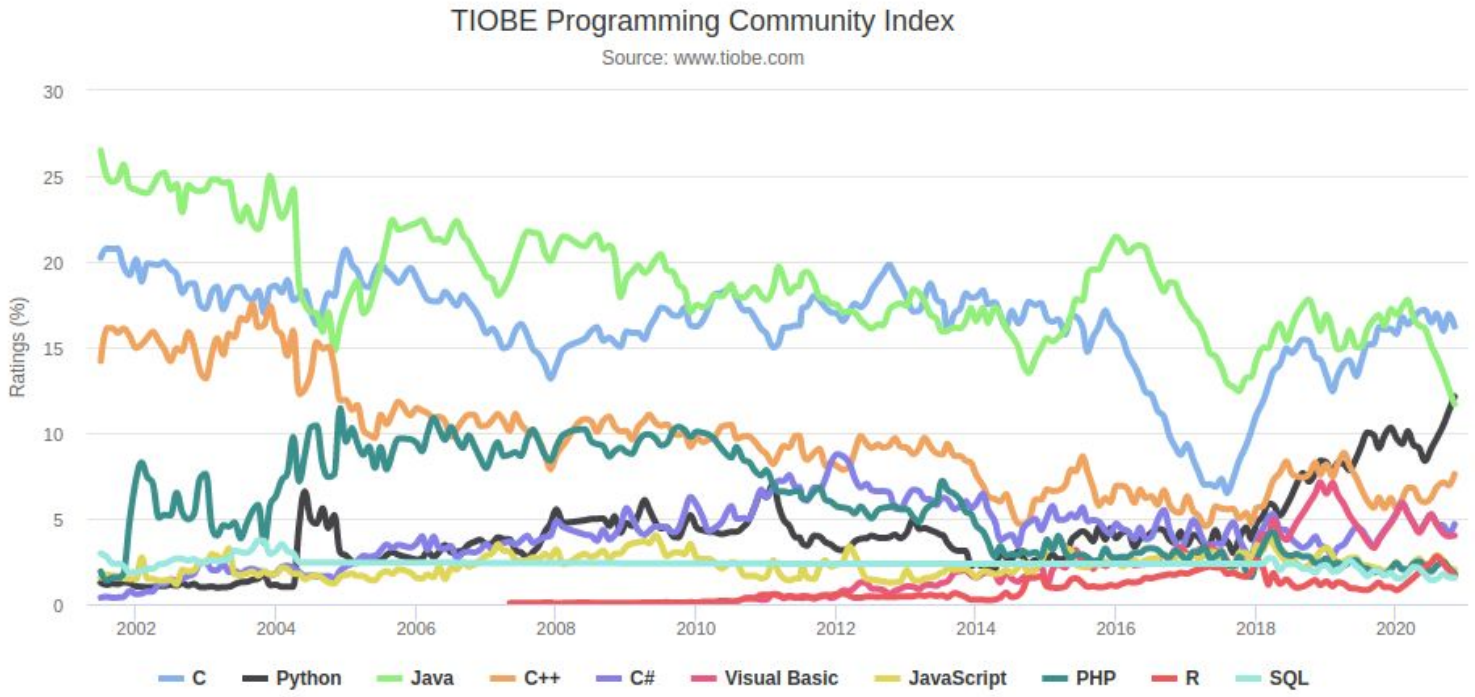
- Compiladas
  - Programa é executado somente após a tradução completa do código fonte
  - Vantagem - eficiência na execução
  - Desvantagem - pode não funcionar em diferentes S.Os. e/ou processadores
  - Fortran, Cobol, C, Ada
- Interpretadas
  - Programa é executado instrução a instrução. Cada comando é traduzido e depois executado
  - Vantagem - rapidez no desenvolvimento
  - Desvantagem - maior tempo de execução
  - Basic, APL, PHP, SQL, Prolog, Haskell
- Híbridas
  - Compilação para um código intermediário e interpretação deste código
  - Vantagem - portabilidade
  - Desvantagem - necessidade de uma máquina virtual
  - Java, C#

Linguagens de programação

	Modelo de execução	Influências	Paradigma principal	Modelo de tipo de dados
C	Compilação	Algol, BCPL	Estruturada, Procedimental, Orientada por fluxo	Estático, fraco
C++	Compilação	C, Simula, Algol 68	Principalmente orientada a objectos, múltiplos paradigmas	Estático, fraco
Objective-C	Compilação	C, Smalltalk	Principalmente orientada a objectos, Reflectiva, Passagem de mensagens	Dinâmico e estático, fraco
Python	Interpretação	ABC, Perl	Orientada a objectos	Dinâmico, forte
Ruby	Interpretação	Smalltalk, Perl	Orientada a objectos	Dinâmico, forte
Mathematica	Interpretação	LISP	Múltiplos paradigmas	Dinâmico, forte
C#	Interpretação e Compilação	Java, C++	Orientada a objectos, múltiplos paradigmas	Estático, forte
Java	Interpretação e Compilação	C++	Orientada a objectos	Estático, forte
Perl	Interpretação	C, Shell, awk, sed, Lisp	Funcional, Orientada a objectos e Procedural	Dinâmico
Boo	Interpretação	Python	Orientada a objectos	Estático
PHP	Interpretação	C e Perl	Orientada a objectos	Dinâmico

Linguagens de Programação

Cenário atual - 2020



https://www.tiobe.com/tiobe-index/

Bibliografia

BROOKSHEAR, J. G. **Ciência da Computação: Uma visão abrangente**. 11a Edição. Bookman, 2013

TUCKER, Allen B.; Noonan, Robert E. **Linguagens de Programação: princípios e paradigmas**. 2ª ed. São Paulo: McGraw-Hill, 2009.

SEBESTA, Robert W. **Conceitos de Linguagens de Programação**. 9ª ed. Bookman, 2010.

Parte do material adaptado das notas da Prof. Fabiane B. Vavassori Benitti